

A new way to measure developer productivity – from the creators of DORA and SPACE

An exclusive interview with the four researchers behind a new developer productivity framework: The three dimensions of DevEx



GERGELY OROSZ

MAY 17, 2023



314



3



26

Share



👋 Hi, this is [Gergely](#) with a bonus, free issue of the Pragmatic Engineer Newsletter. In every issue, I cover challenges at Big Tech and startups through the lens of engineering managers and senior engineers.

If you're not a subscriber, you missed the issue on [Staying technical as an engineering manager](#) and [a few others](#). Subscribe to get two full issues every week. Many subscribers expense this newsletter to their learning and development budget. If you have such a budget, here's [an email you could send to your manager](#). 📌

Despite constant efforts, measuring developer productivity remains a stubbornly elusive goal for organizations. In the newsletter, we've taken deep dives into this topic, as well as into the DORA and SPACE frameworks in recent issues:

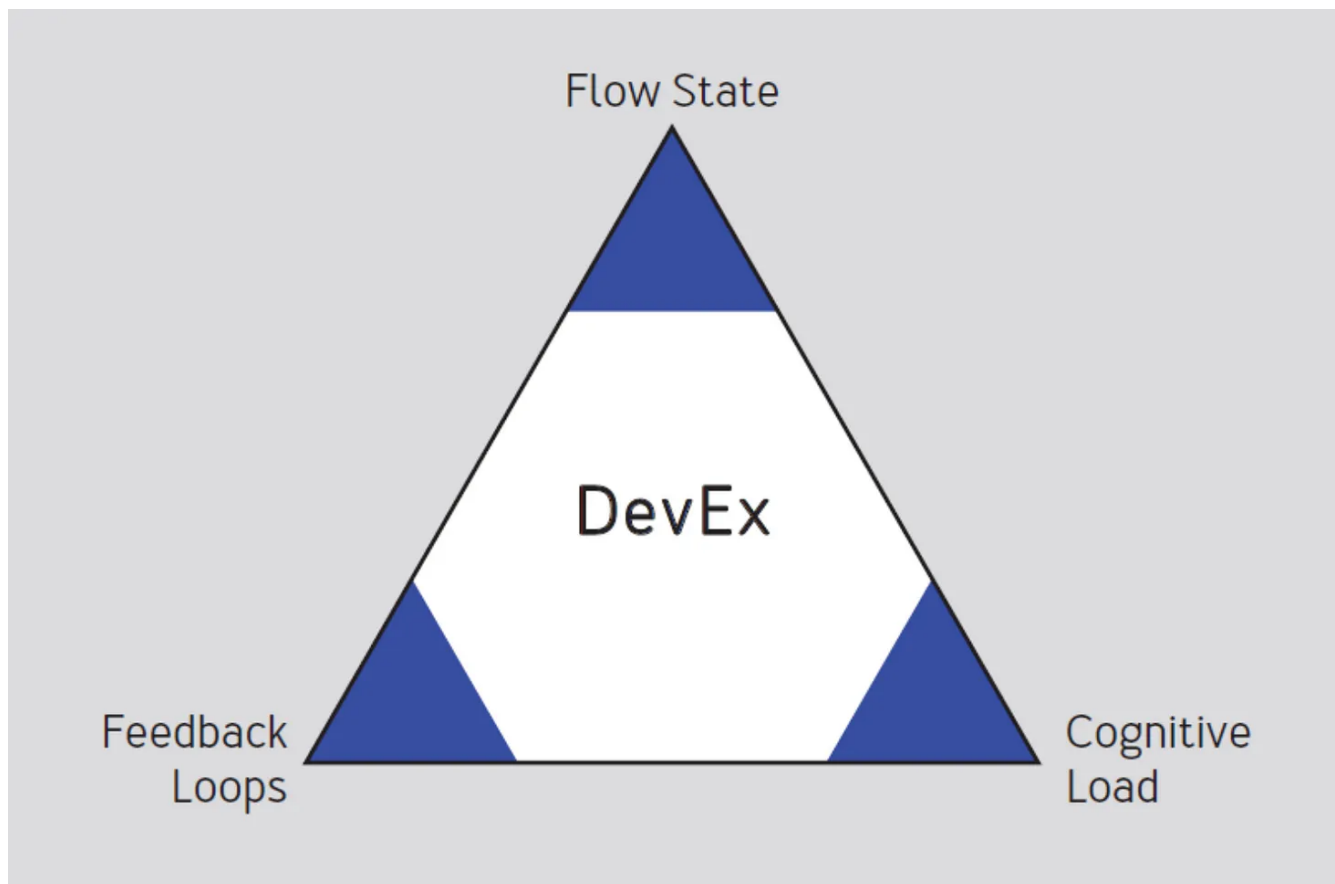
- [Measuring Software Engineering Productivity](#)
- [How LinkedIn measures software engineering efficiency](#)
- [How Uber measures developer productivity](#)

Last November, I gave a talk entitled "[Developer Productivity 2.0](#)" at LeadDev Berlin, in which I tackled what we know about this unexpectedly difficult field.

There are a lot of ideas circulating in this field, but many approaches lack data on whether or not they work, or in which environments they work best. Someone who is

both researching this field and sharing research, is **Abi Noda**, the founder of developer productivity startup, DX (disclosure: I'm [an investor and advisor](#) with DX.) He writes **Abi's Newsletter** which surfaces the latest research and insights on developer productivity, like [the impact of technical debt on developer morale](#).

So when Abi alerted me on how he has teamed up with the lead researchers behind DORA and SPACE to publish a [new framework](#) which they describe as a "developer-centric approach to developer productivity," it piqued my curiosity:



The 3 core dimensions of developer experience. This is a new framework used to think about developer productivity. Source: [DevEx: what actually drives productivity](#) research paper.

I wanted to understand more about this new framework and how organizations can apply it.

In this exclusive interview, the authors take us under the hood to reveal more about this framework and what makes it different from other productivity frameworks.

In this issue, we cover:

1. Introducing the authors of the paper
2. DORA, SPACE and the need for a new approach
3. Developing the new framework
4. Companies which have adopted survey-based approaches
5. How to design effective surveys
6. Other developer productivity papers worth reading
7. Bonus: the biggest challenges with DORA: from Nicole Forsgren, one of the creators of DORA

Here's also a link to the [full paper](#), which was published recently.

I'm [an investor](#) in and advisor to [DX](#), the developer productivity company of which Abi – one of the authors of the paper – is founder and CEO, and where the other authors – Margaret-Anne Storey, Nicole Forsgren, and Michaela Greiler – are advisors. I decided to cover their paper not because of DX, but because the authors also created DORA and SPACE, and I wanted to learn more of their thinking about developer productivity.

1. Introducing the authors of the paper

Abi Noda is CEO and co-founder of DX, a platform for measuring and improving developer experience. Abi's the former founder of developer productivity tool Pull Panda, which GitHub acquired in 2019. At GitHub, Abi worked on tools for helping measure software delivery for GitHub and its customers.

Margaret-Anne Storey is Professor of Computer Science at University of Victoria in Canada, where she researches developer productivity. She is a co-author of the [SPACE Framework](#) – a framework we touch on in [Measuring software engineering productivity](#). She serves as chief scientist at DX and consults with Microsoft on improving developer productivity.

Nicole Forsgren is a partner at Microsoft Research, where she leads the Developer Velocity Lab. She is the former founder of the company DevOps Research and Assessment (DORA,) which Google acquired in 2018, as well as the lead author of the book, [Accelerate: the Science of Lean Software and DevOps](#) and the SPACE Framework.

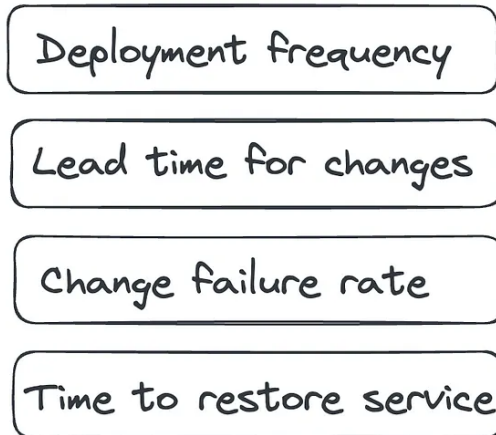
Michaela Greiler is head of research at DX, and was previously a researcher at the University of Zurich and Microsoft Research, where she focused on using engineering data to help improve developer productivity.

Nicole and Margaret-Anne met through Microsoft's [Developer Experience Lab](#) and their work on the SPACE framework. Michaela got to know Margaret-Anne during her PhD studies, while Abi and Nicole know each other from their time at GitHub. Abi started to work with Michaela on developer productivity in 2021. This group – in which plenty of people knew each other already – came together to research a new way to measure developer productivity in 2022.

2. DORA, SPACE and the need for a new approach

DORA SPACE

4 key metrics
published in 2020:



5 dimensions
published in 2021:



pragmaticengineer.com

The DORA and SPACE frameworks

DORA is a framework that has defined the four key metrics which indicate the performance of a software development team:

- Deployment Frequency – How often an organization successfully releases to production
- Lead Time for Changes – The amount of time it takes a commit to get into production
- Change Failure Rate – The percentage of deployments causing a failure in production
- Time to Restore Service – How long it takes an organization to recover from a failure in production

SPACE defines five dimensions to measure, for a more accurate picture of developer productivity:

- **Satisfaction and wellbeing:** how fulfilled developers feel with their work, team, tools, or culture; wellbeing is how healthy and happy they are, and how their work impacts it
- **Performance:** the outcome of a system or process
- **Activity:** a count of actions or outputs completed in the course of performing work
- **Communication and collaboration:** how people and teams communicate and work together.
- **Efficiency and flow:** the ability to complete work or make progress on it with minimal interruptions or delays, whether individually or through a system

The following is an interview, with questions asked by me.

Given Nicole Forsgren was one of the lead developers of DORA, and she and Margaret-Anne D. Storey are co-authors of SPACE, why did you feel there was a need for a new approach?

Margaret-Anne D. Storey: The focus of SPACE was to change how people think and talk about developer productivity. At the time, the industry was thinking about productivity in very narrow ways and we wanted to respond to that.

Many organizations that try to implement SPACE run into challenges because the framework is broad and not easy to apply. For example, I often see companies focusing on metrics like the number of pull requests, which we do not recommend. This is one of the things which motivated us to develop a new approach.

SPACE made the case that organizations must look beyond activity-based measurements like commits and pull requests, and that the perceptions of developers are an especially important dimension to capture. Our new paper provides a practical framework for actually doing this.

After years spent researching and advising companies on how to understand and improve developer productivity, I've come to view developer experience as holding the

key to effective measurement and improvement. With this new framework, I hope to see more leaders and organizations adopt this approach.

Nicole Forsgren: I don't fully agree with the premise of the question. DevEx isn't necessarily a new idea; it's just something that almost no one has paid attention to before, and rarely approached with appropriate rigor, whereas Google and a handful of other top companies have leveraged and benefited from these methods for years. I keep getting asked about the importance of DevEx and how to measure it effectively, so one goal of this paper is to share what we've learned more broadly.

To lay out the existing approaches out in a way that build on each other:

- DORA helps teams and organizations improve their software-driven value delivery, and SPACE provides a framework for improving productivity. (Fun fact: DORA is an instance of SPACE, as a measure of the performance of software.)
- In order to improve the performance of your systems (measured by [DORA 4](#)) most effectively, you should take a [constraints-based](#) approach. I often tell teams there are three good options for doing this: algorithmically (something DORA the company commercially sold,) using something like value stream mapping, or "asking what people are swearing at most often." I say that last one to get some laughs, but it's also true. It's worth pointing out this last one could also be more accurately reworded as "DevEx."
- I'd also like to point out that I've long advocated the importance of using system-based and people-based data in complementary ways, and you can't really improve software delivery or productivity if you fail to take a user-centric view of your system. In software engineering, your users are your developers.

3. Developing the new framework

How have you developed this framework?

Margaret-Anne D. Storey: This work was informed by our previous research and experience, and seeing the gaps in how organizations approach developer productivity and experience. Our goal was to create a practical framework that would be easy for

people to understand and apply, which would capture the most important aspects of developer experience.

Michaela Greiler: In our [prior research](#) (the paper titled [An Actionable Framework for Understanding and Improving Developer Experience](#)), we identified more than 25 top factors affecting developer experience, as well as the organizational challenges and outcomes related to developer experience. Creating this framework was a months-long process of collaborative research and writing. We distilled our earlier research and experience down to only the most practical findings.

In the new paper, you write: “measuring developer experience requires capturing developers’ perceptions – their attitudes, feelings, and opinions – in addition to objective data about engineering systems and processes.”

How did you come to this conclusion? Why has there been so little focus on capturing these hard-to-measure data until now?

Margaret-Anne D. Storey: In our previous paper, [SPACE](#), we stressed the importance of capturing developers’ perceptions – their experiences – in order to construct a complete picture of productivity. Perceptual data that captures these experiences often provides more accurate and complete information, than what can be observed from instrumented system data alone.

For example, system data may be able to tell you how long code reviews take. But without perceptual data, you won’t know whether reviews are actually blocking developers and slowing down development, or whether developers are receiving high-quality feedback.

Abi Noda: Engineering leaders are inclined toward system metrics because they’re used to working with telemetry and log data. However, we cannot rely on this same approach for measuring people.

Measuring developer productivity requires capturing the perspectives and attitudes of developers themselves. Doing this requires expertise in surveys, but the field of psychometrics is often unfamiliar territory for engineering leaders.

Why do you suggest starting off measuring by using surveys?

Nicole Forsgren: We can gather fast, accurate data with surveys: people can tell us about their work experience and systems much faster than we can typically instrument, test, and validate system data. And once we have that cleaned, validated system data, we should still be collecting some surveys, and the best companies do this. As a note, cleaning and validating system data usually takes 1-3 years to really refine if it's done right!

Why do the best companies collect surveys? Because surveys are the only way to gather data about perceptions and behaviors, which reveals important insights into the developer experience.

For example, your telemetry might give you a timeline for your release process. However, only engineers can tell you if it's smooth and painless, which has important implications for burnout and retention. In addition, surveys can provide a holistic view of your systems without the upfront investment of instrumenting and normalizing data across disparate tools.

Michaela Greiler: To add to what Nicole said, some organizations attempt to measure developer experience using system data alone. An example of this is using metrics based on the data stored in their code repositories.

But these organizations are missing a big piece of the puzzle! Metrics about pull requests may offer some visibility of development activities, but very often these metrics can mislead us or distract us from the real opportunities.

4. Companies which have adopted survey-based approaches

Who already utilizes this approach of using surveys to measure some KPIs? And why aren't more companies doing the same?

Abi Noda: Companies like Google, Microsoft, and Spotify have relied heavily on survey-based developer productivity metrics for years. Our company, [DX](#), has helped

hundreds of companies implement this approach – including eBay and Pfizer whom we highlight [in our paper](#) – and we see this trend accelerating, along with the focus on developer experience in general.

One reason this approach has been less widely adopted is that designing and administering surveys is difficult. To put it plainly, running surveys is more difficult than writing SQL queries to produce metrics from GitHub or JIRA data. Running “good surveys” requires expertise and dedicated resources which many organizations don’t currently have.

Margaret-Anne D. Storey: As Abi touched on, designing and running survey programs requires a lot of expertise and investment. We hear of many organizations that run into challenges with designing good surveys, or sustaining good participation rates.

We hope our framework provides a good starting point for leaders to follow, and we plan to continue publishing best practices which others can benefit from.

5. How to design effective surveys

What are some good practices for designing accurate surveys?

Nicole Forsgren: There are a few important steps in good survey writing. Most folks don’t realize how difficult it is to write good survey questions and design good survey instruments. In fact, there are whole fields of study related to this, such as psychometrics and industrial psychology!

A few rules for writing good surveys:

1. Survey items need to be carefully worded and every question should only ask one thing.
2. If you want to compare results between surveys, you can’t change any wording – at all.
3. If you change any wording, you must do rigorous statistical tests.

In survey parlance, we refer to "good surveys" as "valid and reliable," or as "demonstrating good psychometric properties." Developing good survey items requires rigorous design, testing, and statistical analysis:

- **Validity** is the degree to which a survey item actually measures the construct you desire to measure.
- **Reliability** is the degree to which a survey item produces consistent results from your population and over time.

I typically recommend that leaders hire or consult experts in these areas. At the very least, ensure to pre-test surveys with small groups before sending them out to their entire population.

Abi Noda: One way of thinking about survey design that technical folks might appreciate: think of the survey response process as an algorithm that takes place in the human mind.

When an individual is presented a survey question, a series of mental steps take place in order to arrive at a response:

- **Comprehension** of the survey question and instructions, as well as the context.
- **Retrieval** of specific information or memories, and filling in the missing details.
- **Judgment** and interpretation of those thoughts and memories.
- **Mapping** that judgment to a survey response.

Decomposing the survey response process and inspecting each cognitive step can help us refine our inputs to produce more accurate survey results (wow, this sounds like prompt engineering!). There is a lot of great literature on this topic.

How does system-based data fit into designing effective surveys?

Nicole Forsgren: System-based data provides wonderful insights where there's a lot of data, and we need precision. Examples of system-based data include:

- Build and test execution times

- System reliability
- On-call tickets per engineer.

This type of data can give insights into how we are able to deliver value. For example, do our build times detract from or contribute to our velocity? Does our average number of on-call tickets per engineer exceed our support estimates, are we likely to be “failing over” to our engineers and decreasing their productivity?

In addition to insights into the developer experience, survey and system data can provide triangulation opportunities when used to measure key points along the development toolchain. So if developers report information that’s drastically different from the data coming from your systems, this can highlight gaps in the system or problems with your surveys. In cases where there are discrepancies, I’ve found the survey data to be correct more often than not.

6. Further developer productivity papers worth reading

What are some other recent research studies or papers we should be paying attention to?

Michaela Greiler: Many engineering leaders would appreciate Margaret-Anne’s recent research study: [How Developers and Managers Define and Trade Productivity for Quality](#).

This study found developers and managers have differing definitions of productivity, as well as misconceptions about how the other party defines productivity. This highlights the difficulty of trying to pin down a single definition or measure of productivity.

Margaret-Anne D. Storey: I’ve appreciated the new “Developer Productivity for Humans” series by researchers at Google, and in particular their recent article: [A Human-Centered Approach to Developer Productivity](#).

The article examines the inherent difficulties of measuring developer productivity, flaws in most conventional approaches, and the importance of examining both technological and sociological factors.

7. Bonus: the biggest challenges with DORA

[Note from Gergely: Given Nicole is one of the creators of DORA, that DORA is a popular framework within engineering leaders, and Nicole consulted hundreds of companies on DORA, I took the opportunity to ask a question on the minds of many engineering leaders considering using this approach, or already using it:]

Nicole, what are the most common challenges you see organizations facing with the DORA metrics?

Nicole Forsgren: There are two big challenges I see with folks using DORA.

1. Taking a look at the four key metrics, identifying a limitation in their environment, and declaring defeat. For example, I've heard folks say they are working on firmware systems and can't measure one of the endpoints needed for lead time, and therefore can't use the metrics. Or teams on legacy systems proclaim that *obviously* these metrics were created for cloud systems and so they can't use them. Neither of these are true.

The key is to measure, adjust to your context if necessary, understand where you are, and then improve – industry benchmarks be damned!

I've worked with organizations using DORA which are working on air-gapped systems [meaning networks completely locked down from the outside], and teams which are air-lifting software into remote locations.

2. Once a team has measured or benchmarked itself, it's inactionable and people feel stuck. This one, really, is a misconception. At this point, I joke that there's a whole [rest of the book](#) – or research program!

The four metrics are just a signal for “how am I doing?” They’re pretty nice because they’re a quick check that can be hard to game. But then the work begins: what are my constraints or blockers? What are the biggest roadblocks? We can identify them algorithmically, or just start by asking people what they swear at the most often.

By adopting a constraints-based approach, we can improve faster. And by improving our systems with DevEx in mind, we can really amplify the investments and improvements we make.

One of DORA’s limitations is its scope: for a handful of reasons, we focused on the software delivery process – mostly from code commit to release. There’s a lot of important work and systems behavior that happens before and after that which are not captured.

Takeaways

This is Gergely again.

*Many thanks to Abi Noda, Margaret-Anne D. Storey, Nicole Forsgren and Michaela Greiler for sharing their insights. Read the full paper here: [DevEx: what actually drives productivity](#). Do check out the [SPACE Framework](#) and the book [Accelerate](#) for related reading. And subscribe to *Abi's Newsletter* for details on the latest developer productivity research.*

It’s fascinating to see how our understanding of developer productivity keeps evolving, backed by research; research that’s surprisingly hard to do well, meaning less of it takes place.

Developer productivity is more than just “easy to measure” things like the number of pull requests, or even deployment frequency. SPACE was the first in-depth research to confirm that, indeed, productivity is much more nuanced, and many other factors influence productive teams.

Ways to measure developer productivity are not secret inside companies that care about this area; it’s why places like Google or LinkedIn are ahead of the pack; they just

don't necessarily share all of their advancements in public. *We previously covered [how LinkedIn measures engineering efficiency](#).*

It makes sense that you need both "hard" data that's easy to measure, and "soft" data on how developers really feel about their productivity, and what is slowing them down. That we now have a paper that not only proves this, but offers pointers on what good surveys look like, is a win for the industry.

Using surveys is not a new development: some top companies are employing such surveys, alongside measuring other parts of the system. When I was at Uber, the Developer Experience team sent out quarterly surveys to engineers, gathering details on which things worked, and what slowed them down.

If you are not yet surveying developers on their productivity, you are a step behind industry leaders. Of course, designing effective surveys is tricky, and it's worth putting thought into what a final survey will look like. How to design such surveys was outside the scope of the paper, but Nicole Forsgren suggests to pre-test surveys with small groups, at the very least.

I'll close by quoting Nicole Forsgren on the importance of both qualitative and quantitative measurement of developer productivity, and not forgetting who your real "users" are. This insight from the interview resonated the most with me:

"I've long advocated the importance of using system-based and people-based data in complementary ways, and you can't really improve software delivery or productivity if you fail to take a user-centric view of your system; in software engineering, your users are your developers."



314 Likes · 26 Restacks

3 Comments



Write a comment...



Elio Capella May 18

Very interesting read. I missed an example survey which would be helpful to properly convey the idea. This is the first time I learn about SPACE and it seems pretty abstract after reading the article.

♡ LIKE (9) 💬 REPLY ...



Dusko Bajic Writes Crispy Engineering May 23

The more I read about engineering metrics, the less I am interested in practicing any of these. Instead I'd just look at my context and thinking what matters for my org, my engineers, what do I want to accomplish and how can I possible measure it and against which constraints.

2020 - DORA, 2021 - SPACE, 2023 - Something else. To me it looks like that something is off here, as the industry didn't really had a chance to change that much. (we aren't talking about 2000s, 2010s here). If the suggested methods are changing so rapidly, why should we even try applying any of these (considering the org related risks as well).

This is a bit different point of view which is more critical, but I believe it is justified to some extent.

♡ LIKE (3) 💬 REPLY ...

1 reply

1 more comment...

